

Aufgabe 1: Schmucknachrichten

Teilnahme-ID: 76626

Bearbeiter/-in dieser Aufgabe:
Quirin Klemm

29. März 2025

Inhaltsverzeichnis

1	Lösungsidee	2
1.1	Perlen besitzen gleichen Durchmesser	2
2	Umsetzung	4
3	Beispiele	4
4	Quellcode	4

1 Lösungsidee

Eine Nachricht besteht aus mehreren unterschiedlichen Buchstaben. Für jeden Buchstaben ist ein Codewort zu finden, wobei insgesamt ein präfixfreier Code entstehen muss. Dabei soll die Gesamtlänge der codierten Nachricht möglichst gering sein. Daher wird ein präfixfreier Code gesucht, bei dem häufig vorkommende Buchstaben kurze Codewörter erhalten, während seltenere Buchstaben längere Codewörter zugewiesen bekommen. Im Allgemeinen gilt, solange alle Perlen im Durchmesser gleich sind:

$$l = \sum_i p_i \cdot n_i$$

wobei:

- l die Länge der codierten Nachricht ist,
- p_i die Häufigkeit des Buchstabens i in der Nachricht darstellt,
- n_i die Länge des Codewortes für den Buchstaben i ist.

Wenn sich die Durchmesser der Perlen unterscheiden, stimmt diese Rechnung nicht mehr und muss anders berechnet werden.

Um präfixfreien Code zu erzeugen, wird am besten ein Baum benutzt. Da man durch diesen sicher gehen kann, dass keine Codewort der Präfix eines anderen Codewort ist. Dabei ist jedes Blatt des Baums ein Codewort und jede Kante zu dem Blatt eine Code vom Codewort. Der Baum hat mindestens so viele Blätter wie es verschiedene Buchstaben gibt. Jeder interne Knoten hat dabei maximal so viele Kinder, wie es verschiedene Perlen gibt. Die Ebene des Blattes entspricht die Länge des Codewortes.

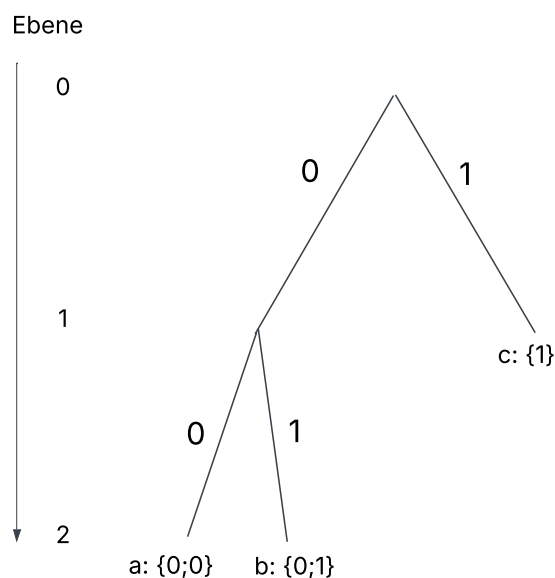


Abbildung 1: simple tree

1.1 Perlen besitzen gleichen Durchmesser

Um möglichst kleinen präfixfreien Code zu erstellen, muss der Buchstabe, der am öftesten vorkommt, ganz oben eingeordnet werden und die nächsten Buchstaben dann in absteigender Reihenfolge. In Abbildung 1 hätte c die höchste Häufigkeit und dannach kommt a und b . Was von a und b die höhere Häufigkeit hat ist egal, da sie beide auf der untersten Ebene sind.

Daher ergibt sich $p_1 \geq p_2 \geq p_3 \dots \geq p_i$ und $n_1 \leq n_2 \leq n_3 \dots \leq n_i$

Zu finden ist also ein Baum, der basierend auf den Häufigkeiten der Buchstaben, seine Blätter so auf seine Ebene so verteilt, dass die Rechnung (1) möglichst gering ist. Jeder interne Knoten muss dabei mindestens zwei Kinder haben, da ein interner Knoten mit nur einem Kind die Äste des Baums verlängern

ohne irgendwelche anderen Vorteile dem Baum zu bringen.

Zu Beginn des Algorithmus wird ein perfekt ausbalancierter Baum erstellt, der so viele Blätter wie Buchstaben enthält. Jedes Blatt wird von links nach rechts in aufsteigender Reihenfolge mit einem Buchstaben belegt, und die Gesamtkosten des Baumes werden gemäß der Formel (1) berechnet. Dabei ergeben sich deutlich höhere Kosten für die rechten Blätter des Baumes im Vergleich zu den linken, da alle Blätter auf derselben Ebene sind, jedoch die Häufigkeit der Buchstaben auf der rechten Seite größer ist. Um das auszugleichen, wird das Blatt mit den höchsten Kosten eine Ebene nach oben verschoben. Dies geschieht indem der Elternknoten des Blattes selber zum Blatt wird und alle Kinder verliert. Dadurch hat der Baum insgesamt zu wenig Blätter, weshalb er erweitert werden muss. Dies geschieht, indem ab der Ebene des entfernten Blattes die Knoten von links nach rechts aufgefüllt werden, bis wieder genügend Blätter vorhanden sind.

2 Umsetzung

Hier wird kurz erläutert, wie die Lösungsidee im Programm tatsächlich umgesetzt wurde. Hier können auch Implementierungsdetails erwähnt werden.

3 Beispiele

Genügend Beispiele einbinden! Die Beispiele von der BwInf-Webseite sollten hier diskutiert werden, aber auch eigene Beispiele sind sehr gut – besonders wenn sie Spezialfälle abdecken. Aber bitte nicht 30 Seiten Programmausgabe hier einfügen!

4 Quellcode

Unwichtige Teile des Programms sollen hier nicht abgedruckt werden. Dieser Teil sollte nicht mehr als 2–3 Seiten umfassen, maximal 10.